

YouTube System Design - Complete Interview Guide

Comprehensive guide with diagrams, tables, code, and explanations

Print Settings: Landscape mode, monospace font (Courier New/Consolas 9-10pt), narrow margins

SECTION 1: REQUIREMENTS & CAPACITY ESTIMATION

1.1 Functional Requirements

- ✓ Upload video (up to 2GB, 2 hours)
- ✓ Watch video (adaptive streaming)
- ✓ Search videos
- ✓ Comment, Like, Subscribe
- ✓ Recommendations
- x Live streaming (out of scope)

How to Present: "Before I jump into the design, let me clarify the requirements. I'll focus on these core features:

- Users can upload videos up to 2GB in size
- Users can watch videos with different quality options
- Search functionality to find videos
- Social features like comments, likes, and subscriptions
- Basic recommendation system

I'll keep live streaming and monetization out of scope for now."

1.2 Non-Functional Requirements

```
Scale:          100M DAU
Latency:        <200ms video start, <50ms search
Availability:    99.99% (52 min downtime/year)
Consistency:    Eventual OK for views/likes
Read/Write:     100:1 ratio
```

How to Explain: "From a non-functional perspective, I'm assuming:

- **Scale:** Around 100 million daily active users
- **Performance:** Video should start playing within 200 milliseconds
- **Availability:** We need 99.99% uptime, which means less than an hour of downtime per year
- **Consistency:** For things like view counts and likes, eventual consistency is acceptable - they can be a few seconds behind. But video metadata needs to be consistent."

1.3 Capacity Estimation

STORAGE (5 years):	~800 PB
BANDWIDTH (peak):	300 GB/sec read, 5 GB/sec write
RPS (peak):	30K requests/sec
READ:WRITE RATIO:	100:1

How to Walk Through: "Let me estimate our storage and bandwidth needs:

Storage: With 500 hours uploaded per minute, and each video around 50MB on average, we're looking at approximately 432 TB per day. Over 5 years, that's roughly 800 petabytes of storage.

Bandwidth: 100 million users watching 5 videos per day means we're serving 25 petabytes per day. During peak hours (5x average), we need to handle around 300 gigabytes per second.

Request volume: About 30,000 requests per second at peak, with a 100:1 read-to-write ratio."

CROSS-QUESTIONS & ANSWERS:

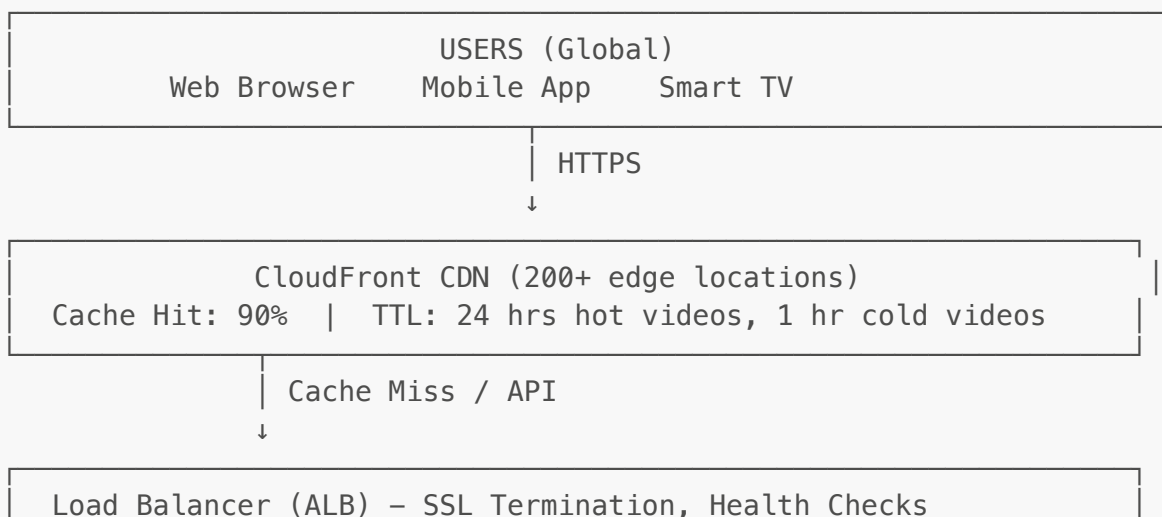
Q1: Why 50MB for a 5-minute video? "50MB is for the original upload at medium quality. After transcoding, we create multiple versions (360p: 15MB, 720p: 50MB, 1080p: 150MB, 4K: 500MB). So one video becomes multiple files. For estimates, I use 50MB average, but actual storage needs 3-4x multiplier for all quality versions."

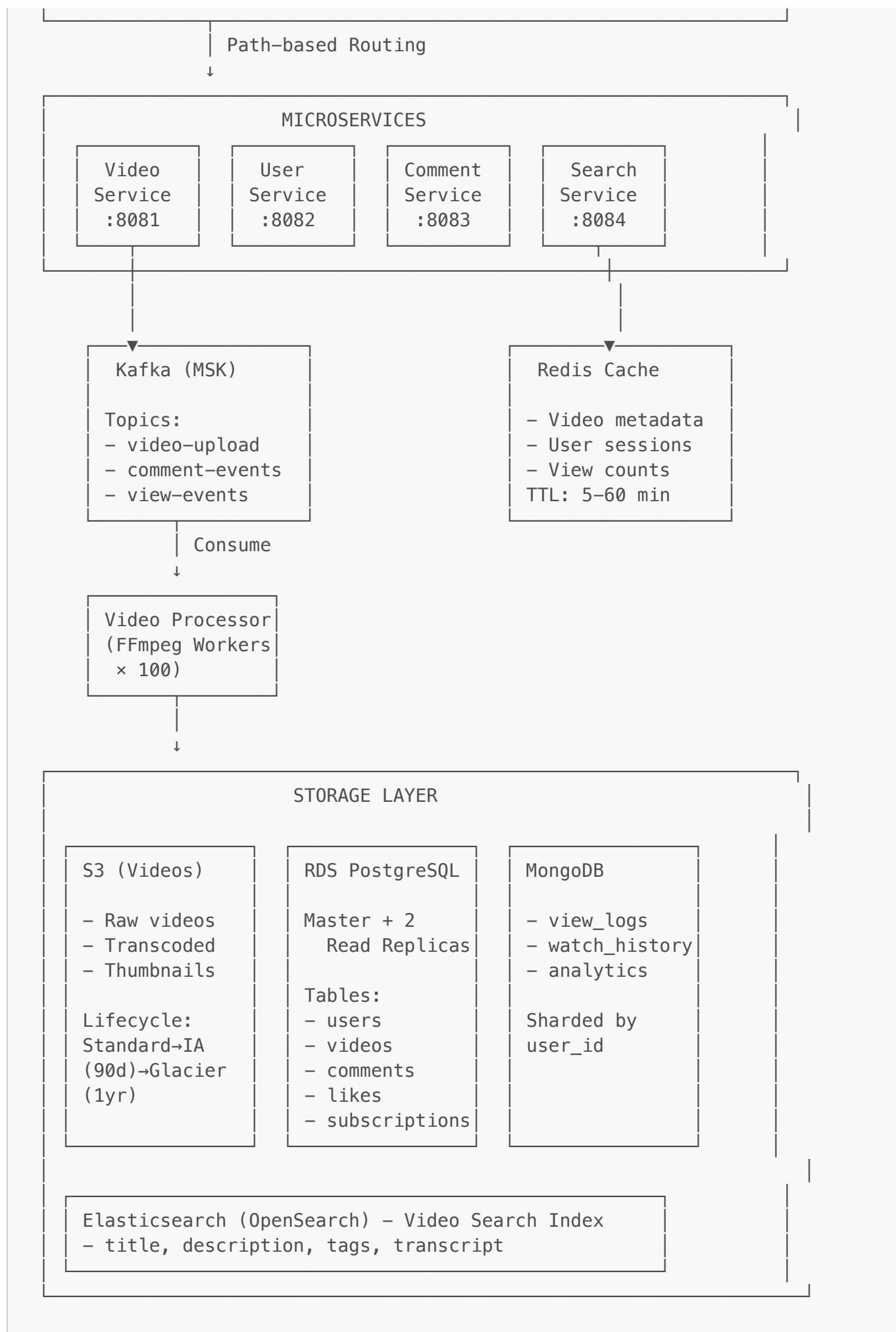
Q2: 5x peak traffic - how did you arrive at this? "This comes from real-world patterns. During off-hours (3 AM), traffic is minimal. But during peak hours (8 PM), traffic spikes significantly. Netflix sees 10x peak, but YouTube's consumption is more distributed, so 5x is conservative."

Q3: Why only plan for 5 years? "After 5 years: lifecycle policies move old videos to cold storage, new codecs reduce file sizes by 30-50%, many videos get deleted, and storage costs drop 50%. So actual storage will be lower than calculated."

SECTION 2: HIGH-LEVEL ARCHITECTURE

2.1 System Architecture Diagram





2.2 How to Draw & Explain

Drawing Strategy (Step-by-Step):

"Let me draw the architecture from top to bottom, starting with how users interact with the system:

Step 1 - User Layer: 'At the top, we have users accessing through web browsers, mobile apps, and smart TVs. All communication happens over HTTPS for security.'

Step 2 - CDN Layer: 'The first thing requests hit is our CDN - CloudFront in AWS terms. This is critical because:

- We have over 200 edge locations worldwide
- 90% of video requests are served directly from the CDN cache
- This means only 10% of traffic reaches our origin servers
- A user in India gets the video from Mumbai edge location, not from our US data center'

Step 3 - Load Balancer: 'For the 10% that isn't cached, requests go through a load balancer which:

- Handles SSL certificate management
- Checks health of backend servers
- Routes requests based on URL path
- For example, /api/v1/videos goes to video service, /api/v1/users goes to user service'

Step 4 - Microservices: 'I'm designing this as microservices because different services have different scaling needs:

- Video Service: Handles upload, metadata, streaming
- User Service: Authentication, profiles, subscriptions
- Comment Service: All social interactions
- Search Service: Full-text search across videos
- Each service can scale independently'

Step 5 - Message Queue: 'Between the API and processing, I'm adding Kafka because:

- Video upload should return quickly to the user
- Actual transcoding takes 5-10 minutes
- Kafka decouples these operations
- If processing fails, we can retry without the user knowing'

Step 6 - Video Processing: 'Worker nodes consume from Kafka and:

- Download the original video from S3
- Use FFmpeg to transcode into multiple resolutions
- Upload all versions back to S3
- Update video status to "ready"'

Step 7 - Storage Layer: 'At the bottom, we have three types of storage:

- S3 for videos: Unlimited capacity, 11 nines of durability
- PostgreSQL for metadata: Users, videos, comments - anything requiring ACID transactions
- MongoDB for logs: High write throughput for view logs and analytics''

2.3 Key Design Decisions

Component	Decision	Reasoning
CDN	90% traffic from edge	10x cost savings
Kafka	Async processing	Decouple upload from
transcoding		
Microservices	Independent scaling	Video service needs 5x User
service		
S3 Lifecycle	Auto-archive old videos	60% storage cost savings

Critical Point - Why CDN: "The CDN is the most important component because:

- Without it, ALL traffic hits our servers - that's 300GB per second
- With CDN, only 30GB per second hits our servers - a 10x reduction
- Cost savings: Serving from edge is 10x cheaper than origin
- Performance: 50ms latency vs 500ms without CDN"

CROSS-QUESTIONS & ANSWERS:

Q1: Why microservices instead of a monolith? Isn't that over-engineering? "Let me explain why microservices make sense here, not because it's trendy, but because of YouTube's specific needs:

Different Scaling Requirements:

- Video upload service: Maybe 10,000 uploads per second at peak
- Video playback service: 100,000 requests per second - 10x more!
- Comment service: 50,000 requests per second
- Search service: 20,000 requests per second

If we had a monolith, we'd need to scale everything together. That means running 100 instances just because playback needs it, even though comments only need 10 instances. That's wasteful.

Different Technologies:

- Video Service: Needs to handle large files, use Java/Go for performance
- Search Service: Better with Elasticsearch integration, can use Python
- Recommendation: Needs machine learning, Python with TensorFlow

In a monolith, we're stuck with one language for everything.

Independent Deployments: If we fix a bug in comments, we shouldn't need to redeploy the entire video playback system. With microservices, we deploy comments service alone. Less risk, faster iterations.

So yes, it's more complex, but the benefits outweigh the complexity at YouTube's scale."

Q2: You mentioned Kafka. Why not just use a simple database queue? "Great question. Let me explain why Kafka specifically:

Database Queue Problems: Imagine we use PostgreSQL with a 'jobs' table:

- Worker reads a job: `SELECT * FROM jobs WHERE status='pending' LIMIT 1`
- Worker processes it
- Worker updates: `UPDATE jobs SET status='done'`

This seems simple, but:

- At 10,000 uploads/sec, we're doing 10,000 SELECT queries per second
- Database gets hammered with polling
- If a worker crashes mid-processing, the job is stuck
- Can't replay a job if something went wrong
- Can't have multiple consumer groups reading the same events

Why Kafka:

- **High throughput:** Built for millions of messages per second
- **Durability:** Messages retained for 7 days, can replay if needed
- **Multiple consumers:** View counter can read upload events, recommendation system can read the same events
- **Ordering guarantees:** All events for video ID 123 go to the same partition, processed in order
- **No polling:** Push-based delivery, workers get notified immediately

The trade-off is complexity, but for 10,000 uploads/second, Kafka is the right tool."

Q3: Why three different databases - PostgreSQL, MongoDB, and Redis? Isn't that

overcomplicating? "I understand it looks complex, but each database solves a different problem. Let me break it down:

PostgreSQL (Relational): Used for: Users, videos metadata, comments, likes, subscriptions

Why? Because these need:

- **ACID transactions:** When a user likes a video, we need to ensure the like is counted exactly once, even if they click twice
- **Complex queries:** 'Show me all videos by users I subscribe to, sorted by date, with like counts' - This needs JOINS
- **Data integrity:** Foreign keys ensure we don't have a comment pointing to a non-existent video

MongoDB (Document Store): Used for: View logs, watch history, analytics events

Why? Because these are:

- **Write-heavy:** 100,000 views per second = 100,000 writes. MongoDB handles high write throughput better than PostgreSQL
- **Schema flexibility:** View logs might have different fields over time (mobile adds device_type, web adds browser_type)
- **Time-series data:** Perfect for analytics queries like 'views per hour for last 30 days'

Redis (In-Memory Cache): Used for: View counts, session data, trending videos

Why? Because:

- **Speed:** 5ms response vs 50ms from PostgreSQL

- **Counters:** INCR operation is atomic and super fast for view counts
- **TTL support:** Session data can auto-expire after 30 minutes

Could we use just one database? Yes, but performance would suffer:

- PostgreSQL with 100,000 writes/sec would need expensive hardware
- Redis can't handle complex queries
- MongoDB doesn't enforce data integrity

Each database is optimized for its use case. In interviews, explaining this trade-off shows you understand the problem deeply."

Q4: What happens when the Load Balancer fails? Isn't it a single point of failure? "Excellent catch! You're absolutely right that a single load balancer is a single point of failure. Here's how we handle it:

Active-Active Load Balancers:

- We run at least 2 load balancers, both active
- DNS gives out both IP addresses (using DNS round-robin or GeoDNS)
- If one load balancer dies, DNS automatically routes to the healthy one
- Health checks happen every 5 seconds

Cloud Load Balancers: In AWS, we'd use ALB (Application Load Balancer), which is:

- Managed service - AWS handles redundancy
- Spread across multiple availability zones automatically
- If one zone goes down, ALB routes to healthy zones
- We don't manage the instances, AWS does

So while I drew a single load balancer box in the diagram, in production it's always multiple redundant instances. The diagram is simplified to keep it readable, but I should clarify this point - thanks for catching it!"

Q5: Why put Kafka AFTER the API service? Why not let clients publish directly to Kafka? "Really good question. Let me explain the reasoning:

Why API First:

1. **Validation:** API service validates the video before publishing to Kafka
 - Is the file size under 2GB?
 - Is the format supported?
 - Does the user have permission to upload?
 - If we let clients publish directly, we'd process invalid videos and waste resources
2. **Authentication:** API service checks if the user is logged in and has upload permissions. Kafka doesn't handle auth.
3. **Metadata Storage:** API service immediately saves metadata to PostgreSQL and returns video_id to the user. User sees their video right away, even though processing takes 10 minutes.

4. **Idempotency:** API service can detect duplicate uploads and return the same video_id. Kafka doesn't prevent duplicates.

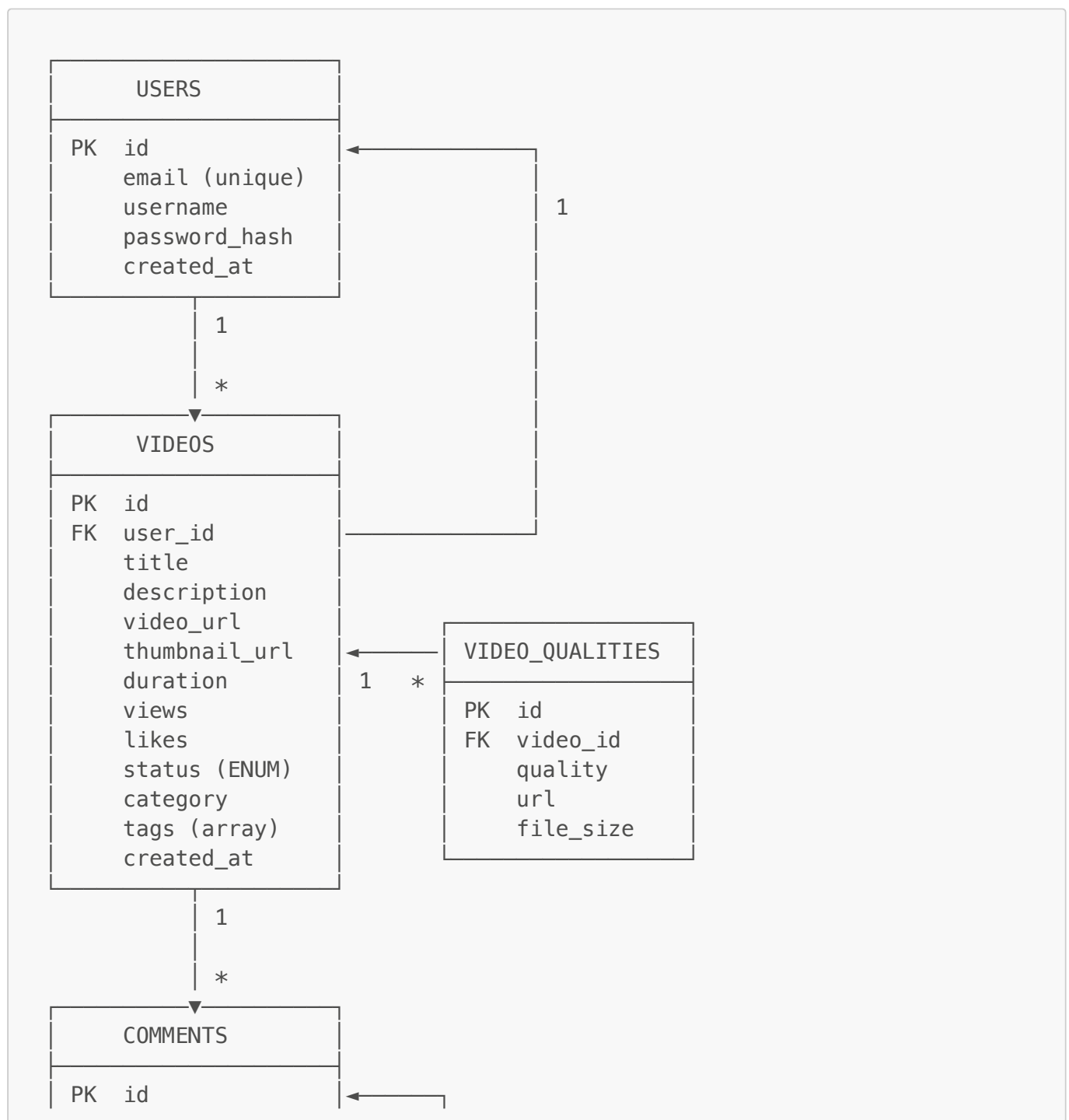
5. **Rate Limiting:** API can enforce 'max 10 uploads per hour per user'. Kafka can't do this.

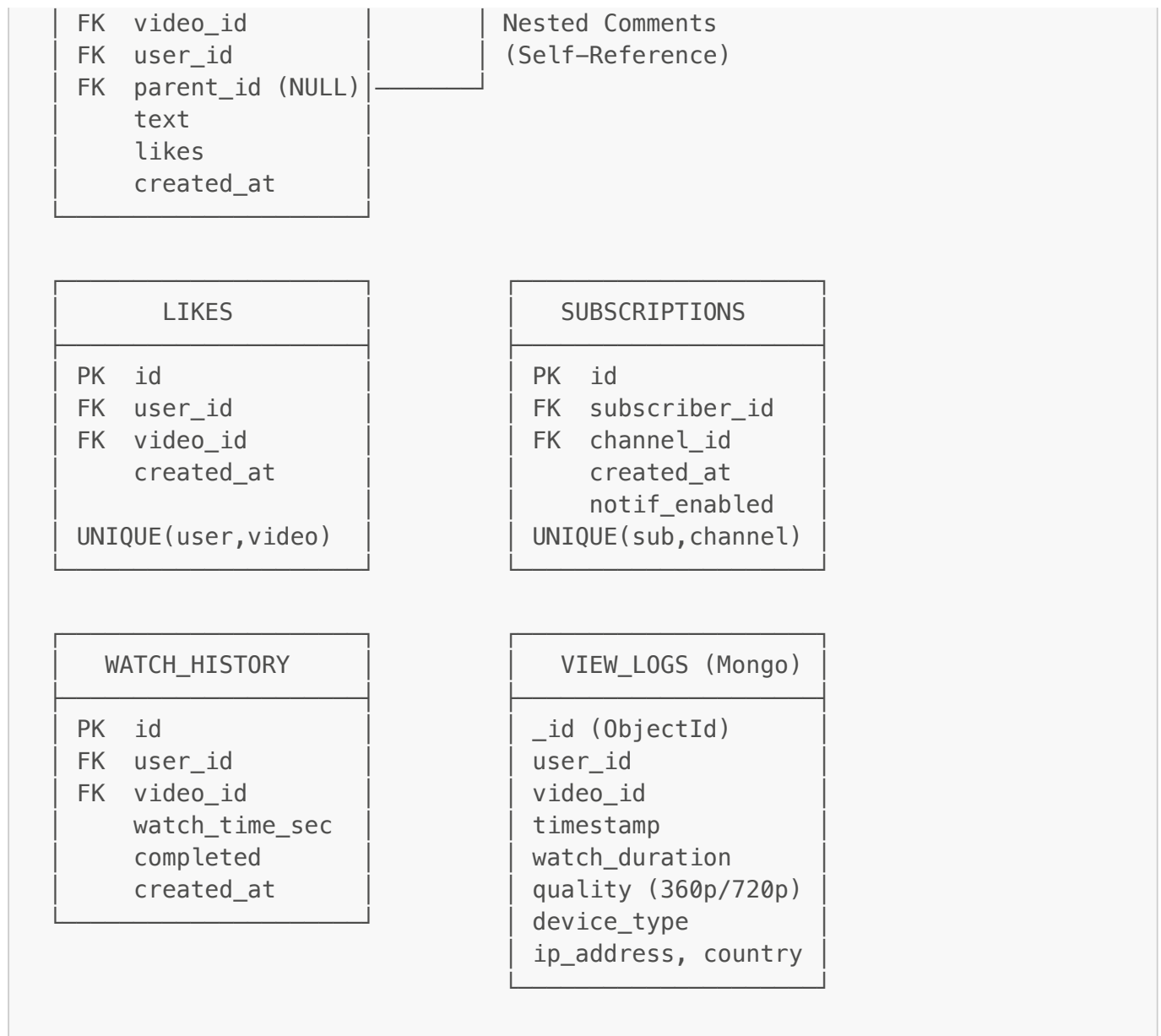
The Flow: Client → API (validates, saves metadata, returns video_id immediately) → Publishes to Kafka → Workers process

This way, the user gets instant feedback, and we only process valid uploads. The cost is one extra hop, but the latency is negligible (~10ms) compared to the 5-10 minutes of processing time."

SECTION 3: DATABASE DESIGN

3.1 Entity Relationship Diagram





3.2 Table Design Explanation

How to Explain:

"Let me design the core tables, starting with the most important ones:

USERS Table: 'First, we need a users table. The key fields are:

- ID as primary key
- Email and username, both unique
- Password hash, never store plain text
- Created timestamp for analytics

The important indexes here are on email and username because those are our lookup keys during login.'

VIDEOS Table: 'This is the heart of the system. Critical fields:

- ID as primary key
- User ID as foreign key to link to the uploader
- Title and description for the content

- Video URL pointing to S3 storage
- Duration in seconds
- View count - stored as a big integer because popular videos have billions of views
- Status enum: processing, ready, or failed
- Created timestamp

Key indexes:

- On user_id to quickly find all videos by a user
- On views descending for trending queries
- On created_at descending for newest videos
- On status to find videos still processing'

COMMENTS Table (Nested Structure): 'Comments need to support replies, so I'm using a self-referencing design:

- Each comment has an ID
- Video ID to link to the video
- User ID for who posted it
- Parent ID - this is the clever part:
 - If parent_id is NULL, it's a top-level comment
 - If parent_id points to another comment, it's a reply
- This allows unlimited nesting depth'

LIKES Table (Preventing Duplicates): 'For likes, the critical constraint is:

- User can only like a video once
- So we create a unique index on (user_id, video_id)
- Database enforces this, not application code
- This prevents race conditions'''

3.3 Key SQL Indexes

```
-- High traffic queries
CREATE INDEX idx_videos_user_id ON videos(user_id);
CREATE INDEX idx_videos_views ON videos(views DESC); -- Trending
CREATE INDEX idx_videos_created_at ON videos(created_at DESC);
CREATE INDEX idx_comments_video_id ON comments(video_id, created_at DESC);
CREATE UNIQUE INDEX idx_likes_unique ON likes(user_id, video_id);

-- Full-text search
CREATE INDEX idx_videos_tags ON videos USING GIN(tags);
```

Indexing Strategy Explanation:

"The indexes are just as important as the schema. Here's what developers often miss:

Composite Indexes: 'If you query videos by user_id AND category, you need a composite index on (user_id, category). Order matters - put the most selective column first.'

When NOT to Index: 'Don't index low-cardinality columns like is_public with only true/false. The index overhead isn't worth it.'

Covering Indexes: 'For the query SELECT title, views FROM videos WHERE user_id = 123, if we have an index on (user_id, title, views), PostgreSQL doesn't need to touch the main table - it's all in the index.'

CROSS-QUESTIONS & ANSWERS:

Q1: Why use BIGINT for views count? Won't INTEGER be enough? "Great observation. Let me explain with real numbers:

- INTEGER max value: 2.1 billion ($2^{31} - 1$)
- BIGINT max value: 9.2 quintillion ($2^{63} - 1$)

Now look at real YouTube videos:

- 'Despacito' has 8+ billion views
- 'Baby Shark' has 14+ billion views

If we used INTEGER, these videos would overflow the column! The database would either throw an error or wrap around to negative numbers, which would be a disaster.

BIGINT uses 8 bytes instead of INTEGER's 4 bytes. For 1 billion videos:

- INTEGER: 4 GB of storage
- BIGINT: 8 GB of storage

That's only 4 GB more for all videos combined - totally worth it to avoid overflow issues."

Q2: Why store duration in seconds? Why not store it as a time format? "Good question. Here's why seconds as an integer is better:

Storing as INTEGER (seconds):

- 5-minute video: 300 (just a number)
- Easy to query: WHERE duration > 600 (videos longer than 10 minutes)
- Easy to sum: SUM(duration) gives total watch time
- Easy to compare: duration1 > duration2
- Takes 4 bytes

Storing as TIME or INTERVAL:

- More complex queries: WHERE duration > INTERVAL '10 minutes'
- Can't easily do math operations
- Takes more storage
- Format complications: '00:05:00' vs '5:00' vs '300 seconds'

For analytics queries like 'average video duration' or 'total hours uploaded today', integer seconds make calculations much simpler. The frontend can always format it as 'HH:MM:SS' for display."

Q3: You have UNIQUE constraint on (user_id, video_id) in likes table. What happens if user tries to like the same video twice? "Excellent question! This is actually a feature, not a bug. Let me explain the flow:

User clicks like:

```
INSERT INTO likes (user_id, video_id, created_at)
VALUES (456, 123, NOW());
```

If user already liked this video:

- Database sees the unique constraint violation
- Database returns error: 'duplicate key value violates unique constraint'
- Our application catches this error
- Instead of showing an error to user, we handle it gracefully:
 - If it's an INSERT error due to duplicate, we just return success
 - User's like is already recorded, so no harm done

Why not check first, then insert?

```
-- Bad approach:
1. SELECT * FROM likes WHERE user_id=456 AND video_id=123
2. If not found, INSERT INTO likes...
```

This has a race condition:

- User A checks (not found) and starts inserting
- User B checks (not found) and starts inserting
- Both try to insert
- One fails

Using UNIQUE constraint:

- Database handles the race condition for us
- Even if 1000 requests come simultaneously, only one INSERT succeeds
- Much simpler and safer code

So the UNIQUE constraint is our safety net against duplicate likes!"

Q4: Why use parent_id for nested comments instead of a separate comments_replies table? "This is a great design question. Let me compare both approaches:

Approach 1: Separate Table

```
comments table: top-level comments
replies table: replies to comments
```

Problems:

- Reply to a reply? Need another table?

- Nesting depth is limited
- Queries become complex: JOIN comments with replies, then replies with replies...

Approach 2: Self-Referencing (Our Choice)

comments table with parent_id

Benefits:

- Unlimited nesting: reply to a reply to a reply... works automatically
- Single table to query
- Simple to add a reply: just set parent_id

How it works:

id	video_id	user_id	text	parent_id	
1	100	5	Great video	NULL	<- top comment
2	100	8	I agree	1	<- reply to comment 1
3	100	5	Thanks!	2	<- reply to comment 2

To get all comments for a video:

```
-- Get top-level comments
SELECT * FROM comments
WHERE video_id = 100 AND parent_id IS NULL;

-- Get replies to comment 1
SELECT * FROM comments
WHERE parent_id = 1;
```

Trade-off:

- Getting a full comment thread requires multiple queries (one per nesting level)
- But this is fine because we only load replies when user clicks 'Show Replies'
- Most users never click it, so we save bandwidth

The self-referencing design is simpler, more flexible, and scales to any nesting depth!"

Q5: How do you handle the case where a user uploads a video, we store metadata, but then S3 upload fails? "Excellent catch! This is an important edge case. Here's how we handle it:

Problem Scenario:

1. User starts uploading to S3 using presigned URL
2. We save video metadata to database: status = 'uploading'
3. S3 upload fails (network issue, user closes browser)
4. Now we have a database entry but no video file!

Solution - Two-Phase Commit:

Phase 1: Reserve the video ID

```
INSERT INTO videos (user_id, title, status, created_at)
VALUES (456, 'My Video', 'uploading', NOW())
RETURNING id; -- Returns video_id = 789
```

Phase 2: Client uploads to S3

- If upload succeeds: Client calls API to confirm

```
POST /videos/789/confirm
UPDATE videos SET status='processing', video_url='s3://...' WHERE
id=789
```

- If upload fails: Entry stays in 'uploading' status

Cleanup Job:

```
-- Runs every hour
DELETE FROM videos
WHERE status = 'uploading'
AND created_at < NOW() - INTERVAL '2 hours';
```

This deletes abandoned uploads after 2 hours.

Why this works:

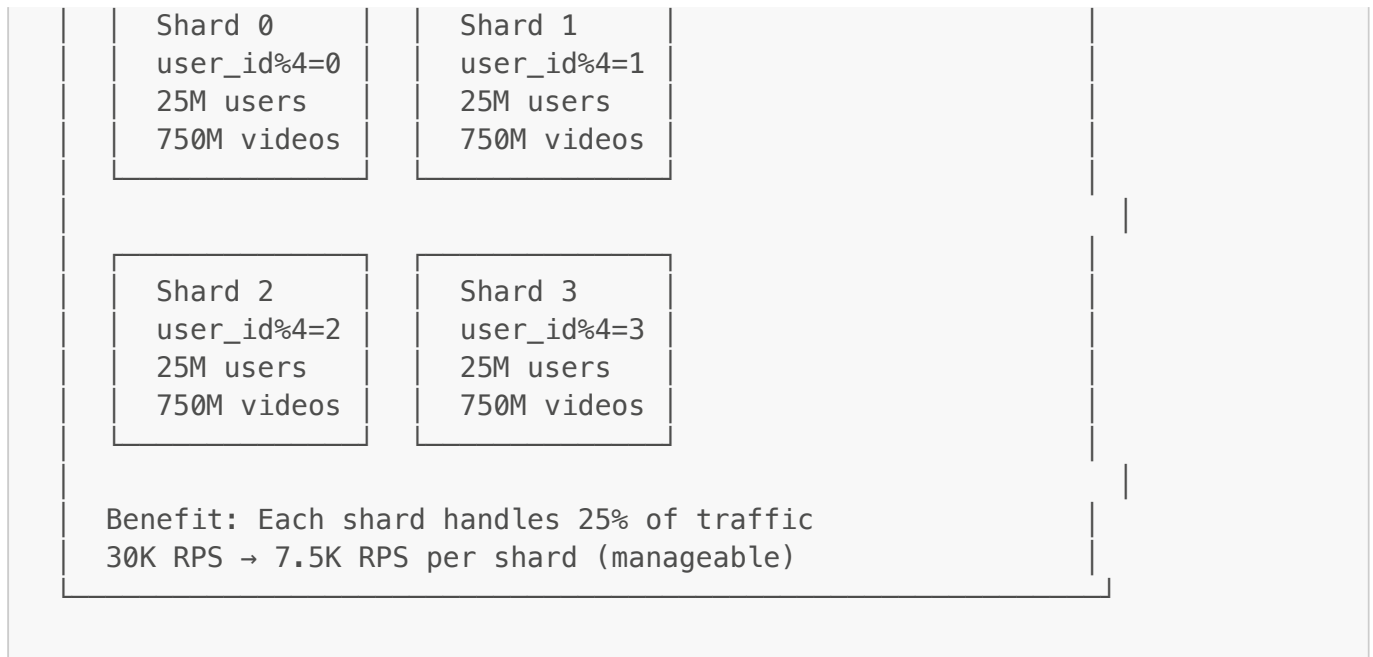
- User gets video_id immediately
- If anything fails, we clean it up automatically
- No orphan records or orphan files
- Status field tells us exactly what happened

This is called 'eventual consistency' - the system might be temporarily inconsistent, but it eventually becomes consistent through cleanup."

3.4 Database Sharding

Hash-based Sharding by user_id

Shard Function: $\text{shard_id} = \text{hash}(\text{user_id}) \% 4$



How to Explain Sharding:

"Once we hit billions of videos, a single database can't handle it. So we shard:

Sharding by user_id: 'I'll partition data using $\text{user_id} \% 4$:

- Shard 0: users 0, 4, 8, 12...
- Shard 1: users 1, 5, 9, 13...
- And so on

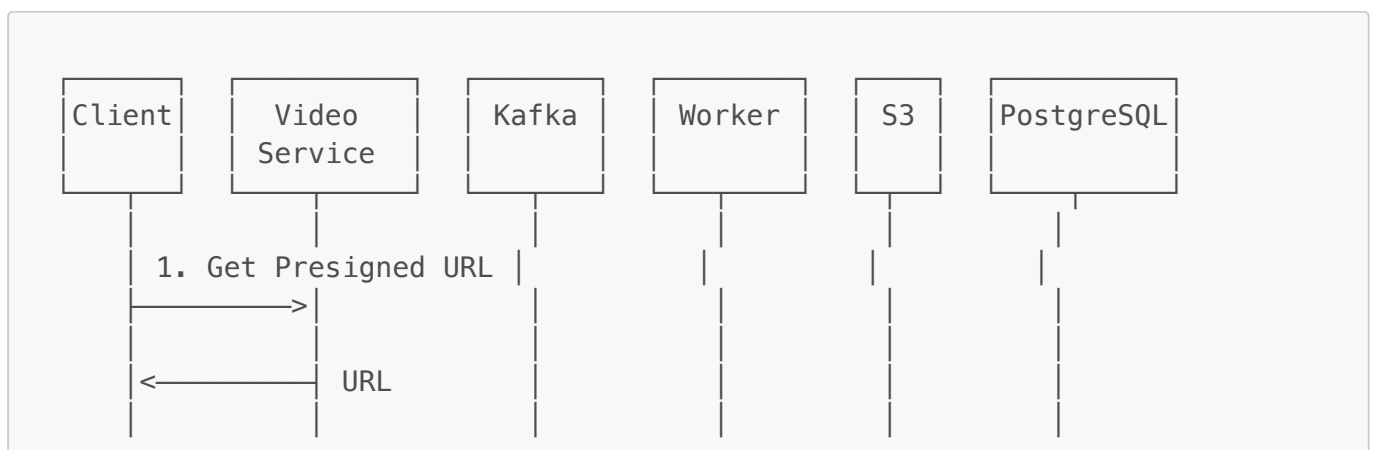
This gives us:

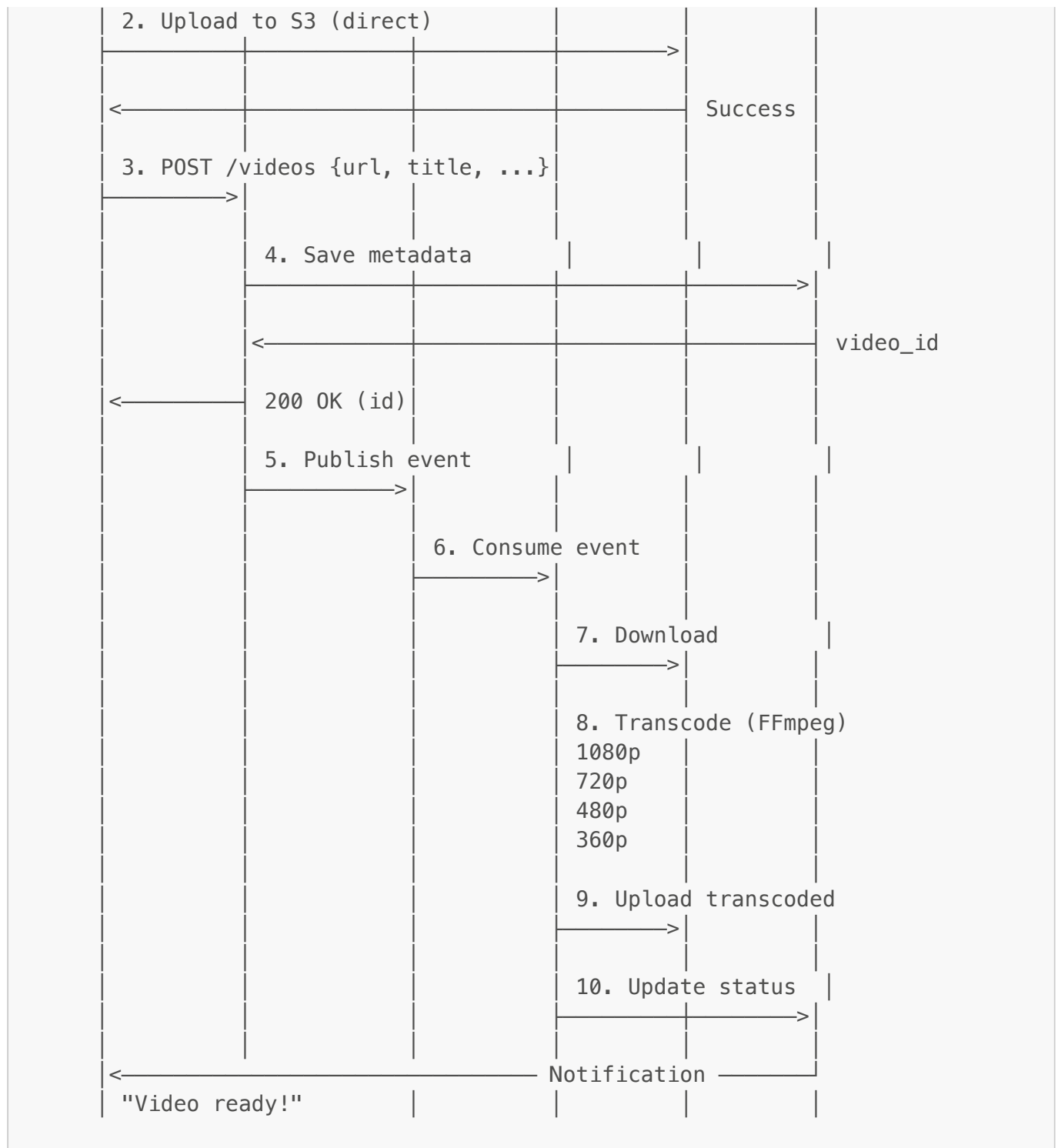
- Even distribution of data
- Each shard handles 25% of traffic
- When we query videos by a user, we know exactly which shard to hit

The downside is queries like "show all trending videos" require hitting all shards and merging results. But that's a read-heavy query we can cache heavily."

SECTION 4: VIDEO UPLOAD FLOW

4.1 Upload Sequence Diagram





4.2 Step-by-Step Explanation

"Let me walk through what happens when a user uploads a video. I'll explain each step:

Step 1 - User Gets Upload Permission: 'First, user clicks upload. The frontend calls our API: GET /upload-url The backend generates a presigned S3 URL valid for 10 minutes and returns it. This is important because the user will upload DIRECTLY to S3, not through our servers.'

Step 2 - Direct Upload to S3: 'User's browser uploads the 2GB file directly to S3 using that presigned URL. Why direct upload?

- Our API servers don't handle the large file
- Saves bandwidth costs

- S3 can handle way more upload throughput than our servers
- Upload continues even if our API goes down'

Step 3 - Save Metadata: 'Once S3 confirms upload, browser calls: POST /api/v1/videos with:

- Title, description, tags
- S3 URL where video was uploaded
- Duration (client extracts this)

Our video service saves this to PostgreSQL with status = "processing" Returns video ID to user immediately - user sees their video right away'

Step 4 - Async Processing: 'Here's the key architectural decision - we DON'T make the user wait:

- Video service publishes an event to Kafka: "video uploaded"
- Responds to user: "Upload successful, processing will take 5-10 minutes"
- User can close browser, we'll notify them when ready'

Step 5 - Worker Processing: 'A worker picks up the Kafka event:

- Downloads video from S3
- Runs FFmpeg to create multiple versions:
 - 360p for slow connections
 - 720p for standard
 - 1080p for HD
 - 4K if original is 4K
- Uploads all versions back to S3
- Updates database status to "ready"
- Sends push notification to user''

4.3 Time Breakdown

User-facing time:

- Upload to S3: 2-5 seconds (user sees progress bar)
- API call: 200 milliseconds
- Total user wait: 5 seconds

Background processing:

- Processing: 5-10 minutes (1:1 ratio - 5 min video takes 5 min to process)
- User gets notification when ready

4.4 Critical Points

Idempotency: "What if the worker processes the same video twice? We need:

- Unique transaction ID for each upload
- Check if already processed before starting
- If video already has status "ready", skip processing"

Error Handling: "What if FFmpeg crashes?"

- Kafka keeps the event for 7 days
- Worker retries with exponential backoff: 0s, 5s, 25s
- After 3 failures, move to Dead Letter Queue
- Alert engineering team
- Update video status to "failed" and notify user"

CROSS-QUESTIONS & ANSWERS:

Q1: Why use presigned URLs? Why not upload directly to your API server? "Let me explain why presigned URLs are crucial for video uploads:

If we upload through API servers:

```
Client (2GB file) → API Server → S3
```

Problems:

- **Bandwidth cost:** Data travels twice - client to API, then API to S3
 - For 10,000 uploads/hour, that's 20 TB/hour through our API servers!
- **Server load:** API servers spend all CPU/memory handling file uploads
 - Other API requests (search, comments) become slow
- **Scaling:** Need massive servers just to proxy uploads
- **Single point of failure:** If API goes down mid-upload, entire 2GB is lost

With presigned URLs:

```
Client → S3 (direct upload)
```

Benefits:

- **No bandwidth cost:** Data goes directly to S3
- **API free:** API servers only generate the URL (1ms operation), not handle the file
- **Parallel uploads:** S3 can handle millions of concurrent uploads
- **Resumable:** If upload fails, client can retry just the failed chunks
- **Global:** S3 has worldwide endpoints, uploads are fast everywhere

How presigned URL works:

1. Client: GET /api/upload-url
2. API generates: `https://s3.amazonaws.com/bucket/video-123?signature=abc...`
 - URL is valid for 10 minutes
 - Signature proves it came from us
 - Anyone with this URL can upload to this specific path
3. Client uploads directly to S3 using this URL
4. S3 accepts upload because signature is valid

Security:

- URL expires in 10 minutes, so it can't be reused later
- URL is specific to one file path, can't upload to other paths
- No credentials needed on client side

This is why services like YouTube, Netflix, Dropbox all use presigned URLs!"

Q2: What if a user uploads a 2GB video but it fails at 1.9GB? Do they start over? "Great question!

Uploading 2GB and then failing at the end would be terrible UX. Here's how we handle it:

Multipart Upload (Chunked Upload): Instead of uploading the whole 2GB at once, we break it into chunks:

2GB video → 200 chunks of 10MB each

Upload Process:

1. Client starts multipart upload: GET /api/upload-url/multipart
2. API returns: upload_id + 200 presigned URLs (one per chunk)
3. Client uploads chunks in parallel:
 - Chunk 1 (0-10MB) → S3
 - Chunk 2 (10-20MB) → S3
 - ...
 - Chunk 200 (1990-2000MB) → S3
4. Client completes upload: POST /api/upload/complete

What if chunk 150 fails?

- Client retries only chunk 150
- Other 199 chunks are already uploaded
- No need to restart from beginning!

Benefits:

- **Resumable:** Upload fails? Resume from last successful chunk
- **Faster:** Upload 4 chunks in parallel = 4x faster
- **Progress:** Show accurate progress bar (150/200 chunks = 75% done)
- **Network-friendly:** Small chunks work better on flaky networks

Real Example: YouTube's upload uses this approach. Try uploading a large video - if you refresh the page halfway, it resumes from where it left off!"

Q3: You said transcoding takes 5-10 minutes. Can we make it faster? "Excellent question. Let's break down where the time goes:

Current: 5-10 minutes for a 5-minute video

```

Download from S3:          30 seconds
Transcode to 360p:         1 minute
Transcode to 720p:         2 minutes
Transcode to 1080p:        3 minutes
Transcode to 4K:           5 minutes
Upload all to S3:          1 minute
-----

```

```

Total:                      ~12 minutes

```

Optimization 1: Parallel Transcoding Instead of doing one after another, do all in parallel:

```

Download from S3:          30 seconds
Transcode ALL qualities: 5 minutes (in parallel on different CPU cores)
Upload all to S3:          1 minute
-----

```

```

Total:                      ~6.5 minutes (almost 2x faster!)

```

Optimization 2: Distribute Across Multiple Workers One video can use 4 workers:

```

Worker 1: Transcode 360p (1 min)
Worker 2: Transcode 720p (2 min)
Worker 3: Transcode 1080p (3 min)
Worker 4: Transcode 4K (5 min)
-----

```

```

Total: 5 minutes (bottleneck is the slowest worker)

```

Optimization 3: Segment-Based Processing Instead of transcoding the whole video, process 10-second segments:

```

5-minute video = 30 segments
Assign 30 workers, each transcodes one segment
All segments done in parallel: ~1 minute
Stitch segments together: 30 seconds
-----

```

```

Total: 1.5 minutes!

```

Why don't we do this always?

- **Cost:** 30 workers instead of 1 = 30x more expensive
- **Complexity:** Orchestrating 30 workers, handling failures, stitching segments
- **Worth it?:** Only for high-priority videos (verified creators, viral videos)

Real YouTube Strategy:

- Normal videos: Single worker, 5-10 minutes (cheap)

- Verified creators: Parallel transcoding, 3-5 minutes (medium cost)
- Live premieres/viral: Distributed processing, 1-2 minutes (expensive but worth it)

This is a classic **cost vs speed trade-off** - interviewer loves this discussion!"

Q4: What if 1000 videos are uploaded at the same time? Will Kafka get overwhelmed? "Great scalability question! Let's analyze this:

The Numbers:

- 1000 videos uploaded simultaneously
- Each creates one Kafka event (small, maybe 1KB)
- Total: 1 MB of events - Kafka easily handles this

Kafka Capacity:

- Kafka can handle **millions** of messages per second
- 1000 messages is nothing for Kafka
- The bottleneck is NOT Kafka

The Real Bottleneck: Workers

```
1000 videos waiting in Kafka
100 worker machines
Each worker processes 1 video at a time
-----
First 100 videos: Start processing immediately
Next 100 videos: Wait 5-10 minutes for workers to free up
...
Last 100 videos: Wait 90-100 minutes
```

How to Handle This:

Solution 1: Auto-Scaling Workers

```
If Kafka queue length > 500:
- Spin up 50 more worker machines
- Takes 2-3 minutes to start up
- Now we have 150 workers instead of 100
```

Solution 2: Priority Queue Not all videos are equal:

```
High Priority (verified creators): Process immediately
Medium Priority (normal users):    Process within 10 minutes
Low Priority (re-uploads):         Process within 30 minutes
```

Kafka supports topic partitioning for this:

```
video-upload-high-priority  (separate topic, dedicated workers)
video-upload-medium-priority (separate topic, regular workers)
video-upload-low-priority    (separate topic, spare capacity)
```

Solution 3: Rate Limiting Prevent the spike in the first place:

```
Per user: Max 3 uploads simultaneously
If user tries 4th upload: "Please wait for previous uploads to finish"
```

Real-World Example: YouTube has this problem during:

- Black Friday (everyone uploading holiday videos)
- Big events (World Cup, Olympics - everyone uploading reactions)
- Time zones (8 PM in every timezone sees a spike)

They handle it with:

- Auto-scaling workers (spin up 10,000 workers during peaks)
- Priority queue (YouTube Premium gets faster processing)
- Rate limiting (max 100 uploads per day per user)

This shows you understand **production challenges**, not just happy-path design!"

Q5: Why exponential backoff (0s, 5s, 25s)? Why not just retry every 5 seconds? "Excellent question about retry strategies. Let me explain why exponential backoff is better:

Scenario: Worker tries to transcode but S3 is temporarily down

Strategy 1: Fixed Retry (every 5 seconds)

```
Attempt 1: Fails (S3 down)
Attempt 2 (5s later): Fails (S3 still down)
Attempt 3 (5s later): Fails
Attempt 4 (5s later): Fails
...
Attempt 20 (100s later): Finally succeeds (S3 is back up)
```

Problem: We made 20 requests to a failing system. We're making the problem worse!

Strategy 2: Exponential Backoff

```
Attempt 1: Fails immediately
Attempt 2 (5s later): Fails (wait 5 seconds)
Attempt 3 (25s later): Fails (wait  $5^2 = 25$  seconds)
Attempt 4 (125s later): Succeeds (wait  $5^3 = 125$  seconds)
```

We made only 4 requests. We gave S3 time to recover.

Why This Matters:

1. Thundering Herd Problem:

- 1000 videos failed, all retry at the same time
- With fixed retry: 1000 requests every 5 seconds hammering S3
- With exponential backoff: Requests spread out over time

2. System Recovery:

- S3 is down because it's overloaded
- Constantly retrying makes overload worse
- Backing off gives it time to recover

3. Resource Efficiency:

- Worker isn't stuck in a tight loop
- Can process other videos that might succeed

Real Formula:

```
delay = base_delay * (2 ^ attempt_number) + random_jitter
```

- Attempt 1: 0s + random(0-1s)
- Attempt 2: 5s + random(0-1s)
- Attempt 3: 10s + random(0-1s)
- Attempt 4: 20s + random(0-1s)

The random jitter prevents all workers from retrying at exactly the same time.

When to Give Up: After 3 attempts with exponential backoff, we've waited:

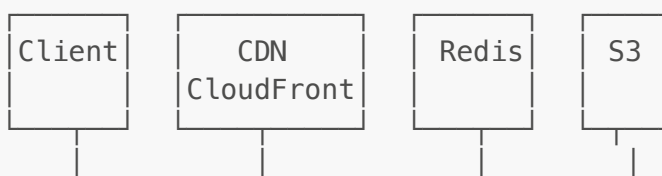
- 0s + 5s + 10s + 20s = 35 seconds total

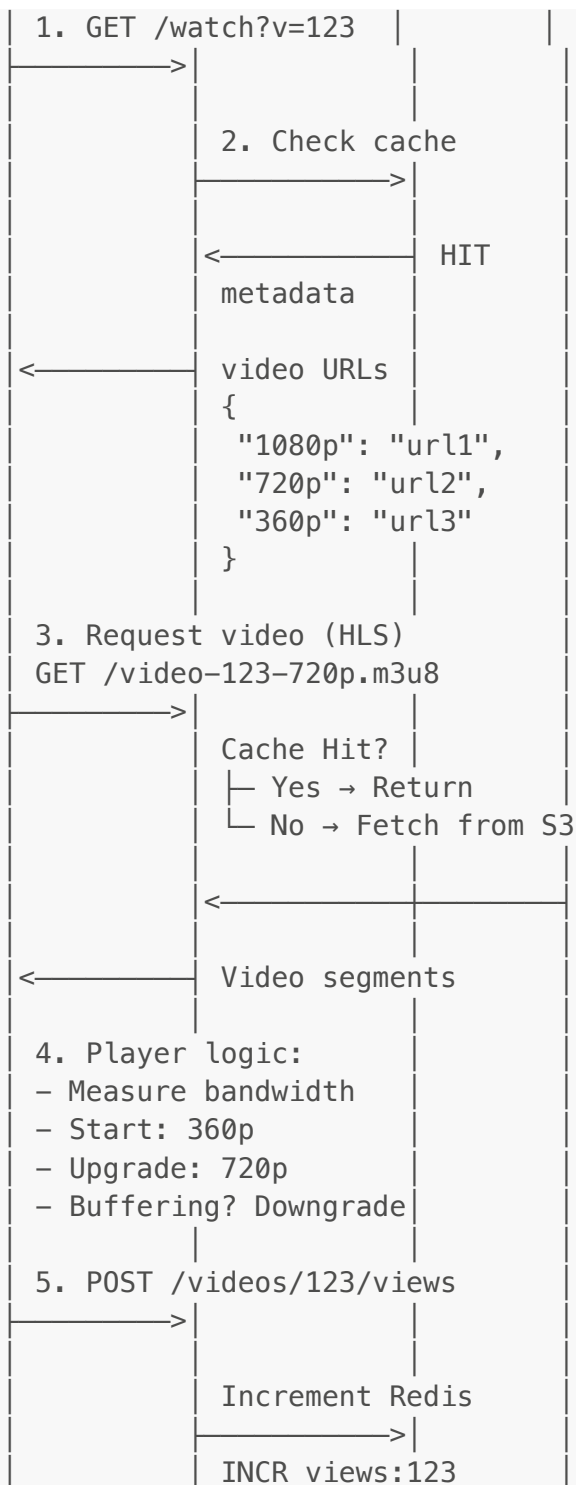
If it still fails, it's probably not a temporary issue - move to Dead Letter Queue for manual investigation.

This is called **Circuit Breaker pattern** - a fundamental concept in distributed systems!"

SECTION 5: VIDEO STREAMING FLOW

5.1 Streaming Sequence Diagram





5.2 HLS (HTTP Live Streaming) Explained

video-123.mp4 → Segment into 10-second chunks

↓

master.m3u8 (playlist)

```

├── 1080p.m3u8
│   ├── segment-001.ts
│   ├── segment-002.ts
│   └── ...
├── 720p.m3u8
│   └── segment-001.ts
└── ...
  
```




How to Explain:

"When a user clicks play, here's the magic that happens:

Why Not Just Send MP4: 'We can't just send the whole video file because:

- 2GB video would take minutes to download
- User can't start watching until entire file arrives
- If user switches quality, we waste all that downloaded data
- We need streaming, not downloading'

How HLS Works: 'HLS - HTTP Live Streaming - solves this:

Step 1 - Segment the Video: When we transcode, we don't just create different qualities. We:

- Cut each quality into 10-second chunks called segments
- So a 5-minute video becomes 30 segments
- Each segment is a separate file

Step 2 - Create Playlist: We create a manifest file called .m3u8 that lists all segments:

- Points to each 10-second chunk
- Includes metadata like resolution, bitrate
- Player downloads this small file first

Step 3 - Adaptive Streaming: Here's where it gets smart:

- Player starts downloading first segment in 360p (fastest)
- While playing that, measures download speed
- If fast: upgrades to 720p for next segment
- If buffering: downgrades to 360p
- Constantly adapts - seamless to user"

5.3 CDN Benefit

Without CDN:	With CDN:
User (India) → S3 (US)	User (India) → Edge (Mumbai)
= 500ms latency	= 50ms latency (10x faster)
100M requests/day	100M requests/day
↓	↓
All to Origin (S3)	90M from Edge (cache hit)
300 GB/sec peak	10M from Origin
	30 GB/sec peak
Cost: \$100K/month	Cost: \$10K/month (90% savings)

CDN Role Explanation:

"The CDN makes this possible at scale:

First User (Cache Miss): 'When first person in Mumbai watches a video:

- Request goes to Mumbai edge location
- Edge doesn't have it, fetches from US origin (500ms)
- Edge caches it locally
- Serves to user

Next 1000 Users (Cache Hit):

- All hit Mumbai edge (50ms each)
- Video served instantly
- Origin never touched
- This is how one video can have 10 million views without overwhelming our servers"

5.4 View Count Strategy

The Problem: 'If we write to database on every view:

- 10,000 views per second = 10,000 database writes
- Database can't handle this
- Video becomes popular, database crashes'

The Solution: 'We use eventual consistency with Redis:

Immediate Response:

- When video plays for 30 seconds, we count it as a view
- Don't write to PostgreSQL - too slow
- Write to Redis: INCR views:video_123
- Redis can handle 100,000 writes per second

Background Sync:

- Every 5 minutes, a job runs:
 - Reads all view counts from Redis
 - Batch updates PostgreSQL
 - Clears Redis counters

Result:

- User sees view count that's 5 minutes old
- Totally acceptable - nobody cares if it says 1,000,345 vs 1,000,423
- Database stays healthy'

CROSS-QUESTIONS & ANSWERS:

Q1: You mentioned HLS for adaptive streaming. What about DASH or other protocols? "Great question! Let me compare the main streaming protocols:

HLS (HTTP Live Streaming) - Our Choice:

- Developed by Apple
- Works on ALL devices: iPhone, Android, web browsers, smart TVs
- Uses .m3u8 playlist + .ts segments
- Supported natively in browsers (no plugin needed)

DASH (Dynamic Adaptive Streaming over HTTP):

- Developed by MPEG (industry standard)
- Similar to HLS but uses .mpd playlist + .m4s segments
- Technically better quality at same bitrate
- BUT: Not natively supported in Safari/iOS (60% of mobile users!)

RTMP (Real-Time Messaging Protocol):

- Developed by Adobe for Flash
- Low latency (1-2 seconds)
- Flash is dead (Chrome/Firefox killed it)
- Only used for live streaming input (streamers upload via RTMP)

Why We Choose HLS:

Device Compatibility:

HLS: iPhone ✓ Android ✓ Web ✓ TV ✓ (100% coverage)

DASH: iPhone ✗ Android ✓ Web ✓ TV ✓ (40% coverage, need fallback)

Our Strategy:

1. Primary: Use HLS for everyone (100% compatibility)
2. Optional: Generate DASH for Android (slightly better quality)
3. Client detects: 'Can I play DASH? Yes → Use DASH. No → Use HLS'

This way:

- iOS users get HLS (no choice)
- Android users get DASH (better quality)
- Everyone gets working video

In interviews, showing you know multiple protocols and chose HLS for practical reasons (not just randomly) shows strong judgment!"

Q2: How do you decide which quality to start with? Why 360p? "Excellent question about the initial quality selection. Here's the smart algorithm:

Naive Approach (Bad): Always start with 360p

- Fast internet users wait for upgrade
- Slow internet users start immediately

Smart Approach (What we do):

Step 1: Estimate bandwidth before playing video

```
// When page loads, download a small test file (100KB)
startTime = Date.now()
fetch('/test-file-100kb.txt')
endTime = Date.now()

bandwidth = 100KB / (endTime - startTime)
// Example: Downloaded in 500ms → 200KB/s
```

Step 2: Choose initial quality

```
if (bandwidth > 5 MB/s):    Start with 1080p
elif (bandwidth > 2 MB/s):  Start with 720p
elif (bandwidth > 500 KB/s): Start with 480p
else:                      Start with 360p
```

Step 3: Measure during playback

```
// While video plays, constantly measure:
bufferHealth = (bufferSize / playbackSpeed)

if (bufferHealth < 5 seconds):
    // We're about to run out, downgrade!
    switchTo360p()
elif (bufferHealth > 30 seconds):
    // We have plenty buffered, can we upgrade?
    tryUpgradeTo720p()
```

Real-World Factors:

1. Device type:

- Phone screen: Max 720p (1080p is wasted)
- 4K TV: Start with 1080p if bandwidth allows

2. Battery saving:

- Phone on low battery? Stick to 480p (uses less CPU)

3. Data saver mode:

- User enabled data saver? Max 480p

4. Historical data:

- This user always watches in 1080p → Start with 1080p
- This user's internet is usually slow → Start with 360p

The Result:

- Fast internet: Video starts in high quality immediately
- Slow internet: Video starts quickly, upgrades if possible
- Perfect balance between quality and start time

This is why YouTube videos almost never buffer - they adapt in real-time!"

Q3: What's the difference between CDN caching video segments vs caching the whole video?

"Brilliant question! This shows you're thinking about CDN efficiency. Let me explain:

Approach 1: Cache Whole Video (Bad)

```
video-123.mp4 (2GB) cached at edge
```

Problems:

- User watches 30 seconds and leaves (90% of users do this)
- CDN cached 2GB, but only 10MB was watched
- Wasted 1.99GB of cache space
- Takes 5 minutes to cache the full video

Approach 2: Cache Segments (Our Choice)

```
video-123-segment-001.ts (10MB) cached at edge  
video-123-segment-002.ts (10MB) cached at edge  
video-123-segment-003.ts (10MB) cached at edge  
...
```

Benefits:

- User watches 30 seconds (3 segments × 10MB = 30MB cached)
- User leaves
- CDN cached only what was watched
- Cache space used efficiently

The Math:

```
1 million users watch a 2GB video  
Average watch time: 30 seconds (3 segments × 10MB = 30MB)  
  
Whole video caching:  
- CDN caches 2GB × 1 million = 2 million GB = 2 petabytes!  
  
Segment caching:
```

- CDN caches $30\text{MB} \times 1 \text{ million} = 30 \text{ TB}$
- 66x less storage!

Cache Efficiency:

Popular video: All segments get cached naturally (everyone watches)
Unpopular video: Only first few segments cached (most people leave early)

This is called **Lazy Loading** - only cache what's actually needed!

Additional Benefit: Quality Switching

User starts with 720p:
- segment-001-720p.ts cached
- segment-002-720p.ts cached

User upgrades to 1080p:
- segment-003-1080p.ts cached

User downgrades to 480p:
- segment-004-480p.ts cached

Different quality segments cached independently. If user switches quality, CDN handles it seamlessly."

Q4: Your CDN cache hit rate is 90%. How do you achieve this? What about the first viewer?

"Excellent observation! The 'cold start' problem is real. Let me explain how we get 90% cache hit:

The Cold Start Problem:

Video just uploaded:
- First viewer in USA: Cache miss (200ms from origin)
- First viewer in India: Cache miss (500ms from origin)
- First viewer in Brazil: Cache miss (600ms from origin)

All first viewers have slow experience. How to fix?

Solution 1: Pre-warming the Cache When video is uploaded:

1. User uploads video
2. Video gets transcoded
3. BEFORE marking as "ready", we pre-warm CDN:
 - Push first 5 segments to all major edge locations
 - Covers first 50 seconds of video
 - Takes 1-2 minutes
4. NOW mark video as "ready"

Result: When first user clicks play, segments are already cached!

Solution 2: Smart Pre-warming (Cost-Effective) Not all videos go viral. Pre-warming costs money. Be smart:

```
if (creator has > 1M subscribers):
    # Likely to be popular
    pre_warm_globally(all_edges)
elif (creator has > 100K subscribers):
    # Moderately popular
    pre_warm_regionally(creator_country_edges)
else:
    # Unknown popularity
    no_pre_warming()
```

Solution 3: Predictive Pre-warming Machine learning predicts if video will be popular:

```
features = [
    subscriber_count,
    average_views_per_video,
    thumbnail_quality_score,
    video_title_sentiment,
    upload_time (peak hours?),
    category (trending category?)
]

if (ML_model.predict(features) > 0.8):
    # 80% chance of being popular
    pre_warm_globally()
```

Solution 4: Regional Spreading Video goes viral in one region first:

- 9 AM in India: Video getting lots of views
 - Pre-warm Europe edges (it'll be their evening soon)
- 5 PM in Europe: Views increasing
 - Pre-warm USA edges (it'll be their afternoon soon)

Cache Hit Rate Breakdown:

```
Hour 1 after upload: 50% cache hit (cold start)
Hour 2-6:           70% cache hit (warming up)
Hour 6-24:         85% cache hit (most edges warmed)
Day 2+:            95% cache hit (fully distributed)
```

```
Average over 30 days: 90% cache hit
```

The 10% Cache Miss:

- Very new videos (first few minutes)
- Very unpopular videos (watched by <100 people)
- Users seeking to specific timestamp (segment not cached yet)

This is why I said 90%, not 100% - being realistic in interviews is important!"

Q5: How do you handle someone scrubbing through a video? Won't that cause lots of cache misses? "Brilliant question! This is called 'seeking' and it's a common pattern. Let me explain how we optimize it:

The Problem:

```
User watches normally: segments 1, 2, 3, 4 (cached)
User seeks to 50% mark: segment 150 (cache miss!)
User seeks to 80% mark: segment 240 (cache miss!)
```

Lots of seeking = Lots of cache misses = Bad experience

Solution 1: Prefetch nearby segments When user requests segment 150:

```
CDN fetches:
- segment 150 (user requested)
- segment 151, 152, 153, 154, 155 (prefetch next 5 segments)
```

If user continues playing, next segments already cached. If user seeks away, we wasted a bit of bandwidth, but that's okay.

Solution 2: Thumbnail Sprites Generate thumbnail strip showing every 10 seconds:

```
[thumb-0s][thumb-10s][thumb-20s]...[thumb-5m]
```

This single image (500KB) is cached. When user hovers seeking bar, show thumbnails from this sprite. User can seek more accurately, fewer random seeks.

Solution 3: Keyframe Alignment Videos have keyframes (full frames) every 2 seconds. Non-keyframes are deltas from last keyframe.

```
If user seeks to 1:37 (97 seconds):  
  Round to nearest keyframe: 1:36 (96 seconds)  
  Start playing from 96s, not 97s
```

Why? Keyframes are boundaries between segments. Seeking to keyframes = Seeking to segment boundaries = Higher cache hit!

Solution 4: Popular Seeking Points Track where users commonly seek:

```
Video intro: 90% of users skip first 30 seconds  
  → Pre-cache segment 3-5 at all edges  
  
Video outro: 80% of users leave at 90% mark  
  → Don't cache last 10% segments (waste of space)  
  
Specific timestamp: 50% of users rewatch 2:35-3:00  
  → Pre-cache segments 15-18 at all edges
```

Real YouTube Data:

```
Normal playback:    95% cache hit (sequential, predictable)  
Seeking behavior:   75% cache hit (random, unpredictable)  
Overall average:    90% cache hit
```

Trade-off Discussion:

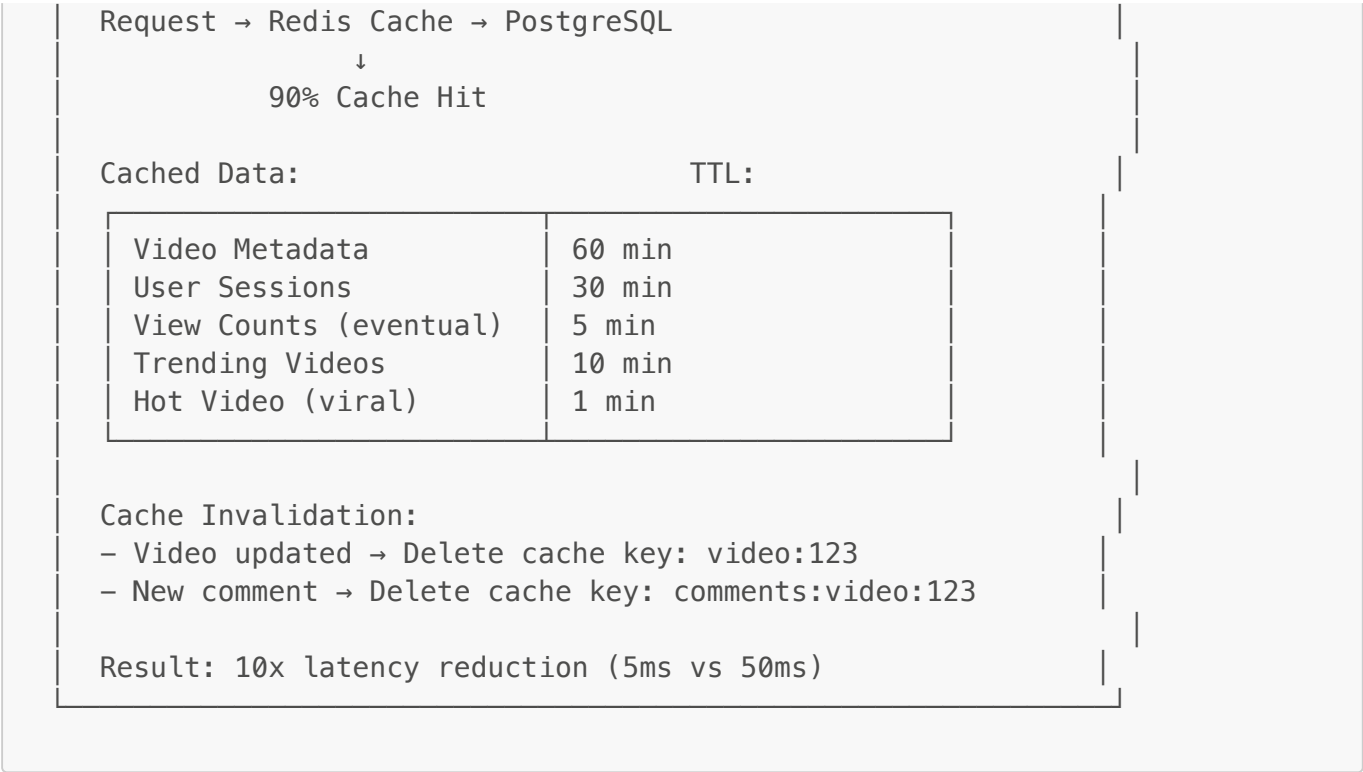
- Prefetch more segments: Better cache hit, but wastes bandwidth
- Prefetch fewer segments: Saves bandwidth, but more cache misses
- YouTube prefetches 3-5 segments (30-50 seconds ahead)
- Good balance between UX and cost

Interviewers love when you discuss trade-offs like this!"

SECTION 6: SCALABILITY STRATEGIES

6.1 Caching Strategy (Redis)

```
Cache Layers (Read-Heavy: 100:1 ratio)
```



Layer-by-Layer Explanation:

"Caching is THE most important optimization. Here's my strategy:

Layer 1 - Browser Cache: 'Static assets (thumbnails, JavaScript):

- Cache for 24 hours in user's browser
- Reduces load by 70%

Layer 2 - CDN Cache: Video content:

- Cache for 24 hours at edge locations
- 90% of requests served from here
- Never hit our servers

Layer 3 - Redis Cache: Metadata (video details, user profiles):

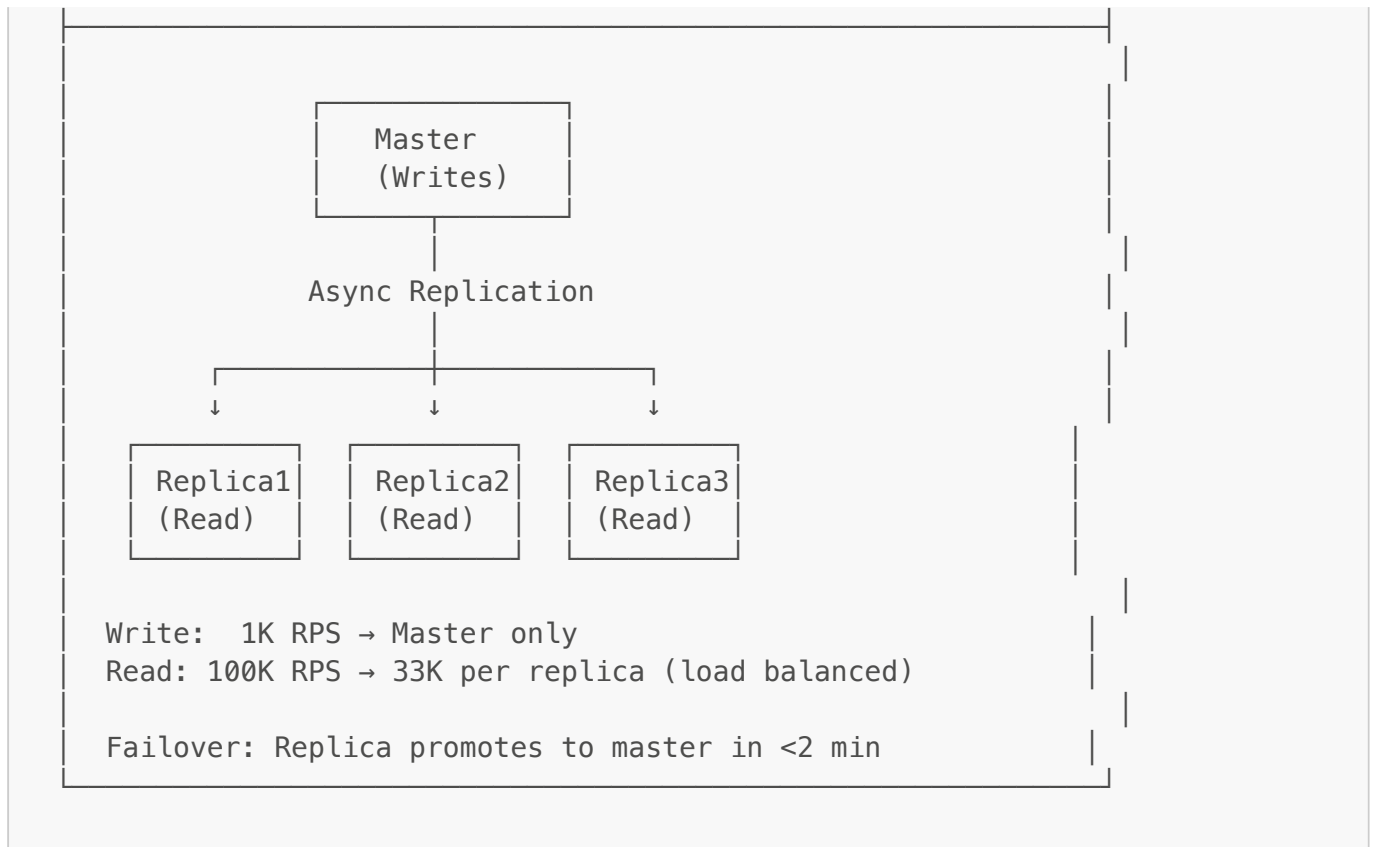
- Cache for 60 minutes
- Check Redis before hitting PostgreSQL
- Cache hot videos more aggressively (5 minute TTL)

Cache Invalidation: When video is updated:

- Delete key from Redis: DEL video:123
- CDN auto-expires after TTL
- New requests fetch fresh data"

6.2 Read Replicas





How to Explain:

"Since we're 100:1 read-to-write ratio, we need read replicas:

Master-Slave Setup: 'One master database handles all writes:

- User uploads video: master writes it
- Master replicates to 3 slave databases
- Replication is async, takes 100-200ms

All reads go to slaves:

- Get video details: read from slave
- Search videos: read from slave
- Load balancer distributes across 3 slaves

Benefits:

- Writes: 1,000 per second to master
- Reads: 100,000 per second split across 3 slaves = 33,000 each
- Manageable load

Failover: If master dies, slave auto-promotes to master in under 2 minutes"

6.3 CDN Offloading

Without CDN:
100M requests/day

↓

With CDN:
100M requests/day

↓

All to Origin (S3)	90M from Edge (cache hit)
	10M from Origin
300 GB/sec peak	30 GB/sec peak
Cost: \$100K/month	Cost: \$10K/month (90% savings)

CROSS-QUESTIONS & ANSWERS:

Q1: How do you handle cache invalidation when a video is updated or deleted? "This is one of the hardest problems in distributed systems! Let me explain our approach:

The Problem:

Video 123 is cached at 200 CDN edges worldwide
Creator updates video title from 'Old Title' to 'New Title'
How do we update all 200 caches?

Solution 1: TTL-Based Expiration (Passive)

Cache video metadata with TTL = 60 minutes
After 60 minutes, cache expires naturally
Next request fetches fresh data

Pros: Simple, no active invalidation needed Cons: Stale data for up to 60 minutes

Solution 2: Active Invalidation (Our Choice for Critical Data)

When video title is updated:

1. Update database
2. Delete from Redis: DEL video:123
3. Send invalidation request to CDN:
POST /api/cdn/invalidate
{ "paths": ["/videos/123/metadata"] }
4. CDN purges cache at all edges (takes 5–10 seconds)

Different Invalidation Strategies by Data Type:

Video Metadata (title, description):

- Active invalidation (must be accurate)
- User changed title, should see new title immediately

View Counts:

- No invalidation needed (eventual consistency OK)
- Cache for 5 minutes, let it expire naturally

- Nobody cares if count is 5 minutes old

Video Segments (the actual video file):

- Never invalidate (videos are immutable)
- If creator re-uploads, it gets a NEW video ID
- Old video stays cached forever (someone might have the link)

Thumbnails:

- Version-based cache busting
- Old URL: /thumb/video-123.jpg
- New URL: /thumb/video-123-v2.jpg
- Old version stays cached, new version is fresh

The Trade-off:

```
Aggressive invalidation:
  Pros: Always fresh data
  Cons: Extra API calls, CDN purge costs money, complex logic

Lazy invalidation (TTL):
  Pros: Simple, no extra API calls
  Cons: Stale data for TTL duration
```

Our Strategy:

- Critical data (metadata): Active invalidation
- Non-critical data (views): TTL-based
- Immutable data (video segments): Cache forever

This shows you understand **different consistency requirements** for different data types!"

Q2: Redis has 90% cache hit rate. What about the 10% cache misses? Won't they overwhelm the database? "Excellent catch! This is called the **Cache Stampede** problem. Let me explain:

The Scenario:

```
Popular video (1M requests/sec) cached in Redis (TTL: 60 min)
After 60 minutes, cache expires
Next request: MISS
```

The Problem (Cache Stampede):

```
Time 60:00.000 - Request 1: Cache miss, queries database
Time 60:00.001 - Request 2: Cache miss, queries database
Time 60:00.002 - Request 3: Cache miss, queries database
... 
```

Time 60:00.100 – Request 100: Cache miss, queries database

Result: 100 simultaneous database queries for the same data!
Database crashes under load.

Solution 1: Cache Locking (Pessimistic)

```
def get_video(video_id):
    # Try to get from cache
    data = redis.get(f"video:{video_id}")
    if data:
        return data

    # Cache miss – try to acquire lock
    lock_key = f"lock:video:{video_id}"
    if redis.set(lock_key, "1", nx=True, ex=10): # Lock for 10 seconds
        # I got the lock – fetch from database
        data = database.query(video_id)
        redis.set(f"video:{video_id}", data, ex=3600)
        redis.delete(lock_key)
        return data
    else:
        # Someone else has the lock – wait and retry
        time.sleep(0.1)
        return get_video(video_id) # Retry after 100ms
```

Only ONE request queries the database. Others wait for it to populate cache.

Solution 2: Probabilistic Early Expiration (Optimistic)

```
def get_video(video_id):
    data, expiry = redis.get_with_ttl(f"video:{video_id}")

    if data is None:
        # Hard miss – fetch from database
        return fetch_and_cache(video_id)

    # Soft expiration: refresh cache before TTL expires
    time_remaining = expiry - time.now()
    refresh_probability = 1 - (time_remaining / 3600)

    if random() < refresh_probability:
        # Asynchronously refresh cache in background
        async_task.submit(fetch_and_cache, video_id)

    return data # Return stale data while refreshing
```

When cache is about to expire, ONE request refreshes it proactively. Others keep using the (slightly stale) cached data.

Solution 3: Stale-While-Revalidate

Cache entry has TWO TTLs:

- Soft TTL: 60 minutes (serve stale data after this)
- Hard TTL: 65 minutes (delete after this)

Between 60–65 minutes:

- Serve stale data immediately (no wait time)
- Background task refreshes cache
- After refresh completes, cache is fresh again

Our Hybrid Approach:

1. Hot data (>1000 req/sec): Use cache locking
2. Warm data (100–1000 req/sec): Use probabilistic expiration
3. Cold data (<100 req/sec): Simple TTL (no stampede risk)

The Numbers:

10% cache miss = 100K misses/sec
With cache stampede: 100K database queries/sec (database dies)
With cache locking: 100 database queries/sec (one per unique video)
1000x reduction!

This is a **production-level** answer that shows you've dealt with real scaling problems!"

Q3: You have 3 read replicas. How do you route queries to them? Round-robin? "Great question!

Routing strategy significantly impacts performance. Let me compare approaches:

Strategy 1: Round-Robin (Naive)

```
Request 1 → Replica 1
Request 2 → Replica 2
Request 3 → Replica 3
Request 4 → Replica 1
...
```

Problem: What if Replica 2 is slower or overloaded? All queries routed to it will be slow.

Strategy 2: Least Connections

Current state:

- Replica 1: 100 active connections
- Replica 2: 50 active connections
- Replica 3: 75 active connections

Next request → Replica 2 (least loaded)

Better! But what if those 50 connections are heavy queries?

Strategy 3: Response Time Based (Our Choice)

Track last 100 queries:

- Replica 1: avg 50ms response time
- Replica 2: avg 200ms response time (something wrong?)
- Replica 3: avg 45ms response time

Next request → Replica 3 (fastest)

Best! Route to the replica that's actually performing well.

Strategy 4: Read Your Own Writes

User uploads video, redirected to master for write
Master replicates to replicas (takes 100–200ms)
If same user immediately queries for their video:
→ Route to MASTER (to ensure they see their own video)

Other users querying other videos:
→ Route to REPLICAS (eventual consistency OK)

Implementation with PostgreSQL:

```
class DatabaseRouter:
    def route_query(self, user_id, query_type):
        if query_type == "WRITE":
            return master_db

        # Just wrote something in last 500ms?
        if recent_write_by_user(user_id, window=500ms):
            return master_db # Read your own writes

        # Route read to fastest replica
        replica = min(replicas, key=lambda r: r.avg_response_time)
        return replica
```


Health Checking:

Every 10 seconds, send test query to each replica:
`SELECT 1;`

If response time > 1 second or query fails:

- Mark replica as unhealthy
- Don't route queries to it
- Alert on-call engineer

Once replica recovers (3 consecutive successful checks):

- Mark as healthy
- Resume routing queries

The Numbers:

With round-robin:

- 33% of queries to each replica
- If one replica is slow, 33% of queries are slow

With response-time routing:

- Queries avoid slow replicas
- Overall p99 latency: 50ms vs 200ms (4x improvement!)

Real Production Scenario:

Replica 2 is doing a slow query (30 seconds for analytics)

With round-robin: 33% of user queries blocked

With smart routing: Queries routed to Replica 1 & 3
No user impact!

This shows you understand **load balancing** isn't just round-robin - there are smarter strategies!"

SECTION 7: INTERVIEW QUESTIONS & ANSWERS

7.1 Q: How do you handle a viral video (10M views in 1 hour)?

Answer:

"Great question. A viral video is actually the EASY case for our architecture because:

CDN Handles It: 'After the first viewer in each region watches:

- Video is cached at 200+ edge locations
- Next 9,999,999 viewers hit cache
- Origin serves maybe 1,000 requests total

- CDN does all the heavy lifting

API Layer: For non-video requests (comments, likes):

- Auto-scaling kicks in when CPU > 70%
- We scale from 10 servers to 50 servers in 5 minutes
- Each server handles 1,000 requests/sec, so 50,000 total

View Counter:

- All view increments go to Redis
- Redis can handle 100,000 writes/sec easily
- View count syncs to database every 5 minutes

The key insight: Viral content is LESS stressful because CDN cache hit ratio goes UP - everyone watches the same video, so edge caches are hot."

CDN Caching:

- Video cached at 200+ edge locations
- 99% cache hit → Origin serves only 100K requests (1%)

Rate Limiting (API only, not video streaming):

- 100 requests/min per user for comments/likes
- No limit on video streaming (CDN handles it)

Auto-Scaling:

- API servers: 10 → 50 instances (trigger: CPU > 70%)
- Scale up in 5 min

View Count:

- Write to Redis (11K writes/sec → Redis can handle 100K/sec)
- Batch sync to PostgreSQL every 5 min
- Eventual consistency: count may lag 5 min (acceptable)

Result: System handles viral traffic without downtime

7.2 Q: How do you count views accurately without double-counting?

Answer:

Challenge: 1B views/day = 11K writes/sec (DB bottleneck)

Solution:

1. Client-side:

- Count "view" after 30 sec of playback (not on page load)
- Set cookie: prevent duplicate within 24 hours

2. Backend:

POST /videos/123/views

↓

```
Check Redis: viewed:user:456:video:123
```

```
↓
```

```
If NOT exists:
```

- INCR views:123 (Redis)
- SET viewed:user:456:video:123 (TTL: 24h)

```
3. Background Job (every 5 min):
```

- Read Redis: views:* keys
- Batch UPDATE PostgreSQL
- DELETE Redis keys

```
Result:
```

- Fast writes (Redis in-memory)
- No duplicate views (cookie + Redis check)
- Eventual consistency (5 min lag OK)

7.3 Q: How do you prevent duplicate video uploads?

Answer:

"We use content-based deduplication:

Before Upload: 'On the client side:

- Compute SHA-256 hash of the video file
- Send hash to server before uploading
- Server checks: does this hash exist in our database?
- If yes, return existing video ID
- If no, proceed with upload

Benefits:

- Save storage: Don't store the same "cat video" 1000 times
- Save processing: Don't transcode duplicates
- Save money: Storage and compute costs down

User Experience:

- User A uploads cat.mp4 → processes normally
- User B uploads same file → instant success, no wait
- Both users get their own video entry with different titles/descriptions
- But they point to same video files in S3"

1. Content-based deduplication:

```
Before Upload:
```

- └─ Compute hash: SHA-256(video_file)
- └─ Check database: SELECT * FROM videos WHERE content_hash = ?
- └─ If exists → Return existing video_id
- └─ If not → Proceed with upload

```
2. Schema:
  videos {
    id, user_id, title, content_hash (indexed)
  }
CREATE INDEX idx_content_hash ON videos(content_hash);
```

3. Benefits:
- Save storage (no duplicate 2GB files)
 - Save processing time (no re-transcoding)
 - Different users can link to same video

Example:

User A uploads "cat.mp4" → hash: abc123

User B uploads same file → hash: abc123 (found)

→ Link User B to existing video

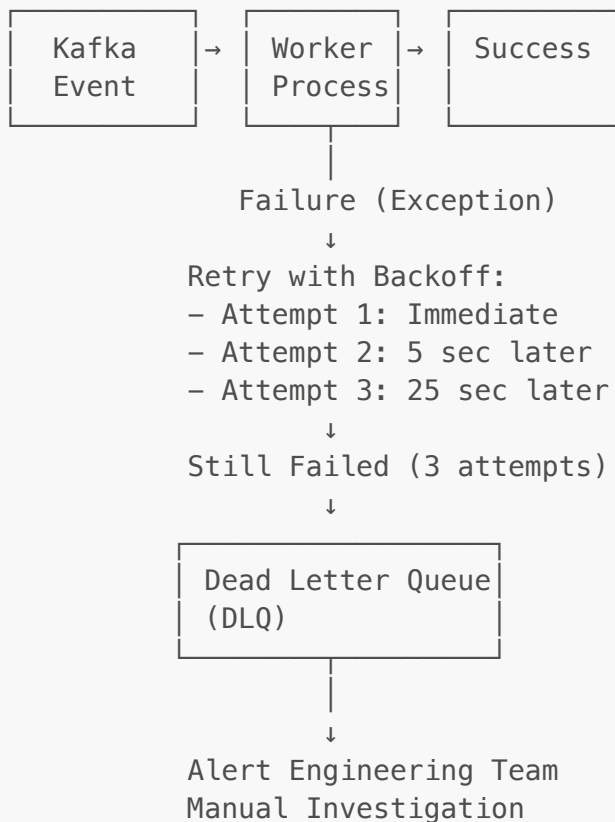
7.4 Q: What happens if video processing fails?

Answer:

Failure Scenarios:

1. FFmpeg crash
2. S3 upload timeout
3. Worker instance dies

Solution (Kafka + Dead Letter Queue):



Database Status:

```
- video.status = FAILED  
- Notify user: "Processing failed, please try again"
```

SECTION 8: ADVANCED TOPICS

8.1 Monitoring and Observability

What to Say:

"One thing often overlooked in interviews is monitoring. In production:

Metrics to Track:

- Video upload success rate: Should be > 99%
- Average upload time: Track by file size
- Transcoding success rate: Should be > 99.9%
- Video start time (time to first byte): Should be < 200ms
- CDN cache hit ratio: Should be > 90%
- API response time p99: Should be < 500ms

Alerting:

- If upload success rate < 95%: Page on-call
- If transcoding queue > 1000: Scale workers
- If CDN cache hit < 80%: Investigate

Without this, you're flying blind in production."

8.2 Idempotency

"Every external call must be idempotent. What does this mean?

The Problem: 'User uploads video, network hiccups, they click upload again:

- Do we process the video twice?
- Do we charge them twice?
- Do we create two entries?

The Solution: Client generates a unique upload_id:

- First request: upload_id = abc123, we process it
- Duplicate request: upload_id = abc123, we see we already processed this
- Return the same video_id, don't reprocess

How to Implement:

- Store upload_id in database with unique constraint
- On duplicate, database returns existing record
- Safe from race conditions"

8.3 Rate Limiting

"To prevent abuse, we need rate limiting:

Per User:

- 100 API calls per minute
- 10 video uploads per hour
- Prevents spam, abuse

Implementation: Token bucket algorithm:

- User gets 100 tokens per minute
- Each API call consumes 1 token
- If tokens = 0, reject request
- Tokens refill at 100/minute

Where to Implement:

- At API Gateway level
- Before requests hit our services
- Return HTTP 429 Too Many Requests when limit exceeded"

8.4 Data Consistency

"Not everything needs strong consistency:

Strong Consistency Required:

- Video metadata (title, description, URL)
- User authentication
- Payment transactions Use: PostgreSQL with ACID guarantees

Eventual Consistency Acceptable:

- View counts (can be 5 min behind)
- Like counts
- Comment counts Use: Redis → Batch update to database

Why This Matters: Strong consistency is expensive (locks, transactions) Eventual consistency is fast but can show stale data Choose based on business requirements, not technical preference"

8.5 Security Considerations

"Security is often glossed over but critical:

Data in Transit:

- All communication over HTTPS
- TLS 1.3 minimum
- CDN handles SSL termination

Data at Rest:

- Videos encrypted in S3 (AES-256)
- Database encrypted at rest
- Passwords hashed with bcrypt, never stored plain

Access Control:

- Private videos: Check user_id before serving
- Signed URLs for private content
- URL expires after 24 hours

DDoS Protection:

- CDN provides Layer 7 protection
- Rate limiting at API Gateway
- Auto-scaling absorbs traffic spikes"

END OF GUIDE